


# 计算思维与人工智能

## 实验三 指针与数组

GO!
TAKE YOU ON A TRIP

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止

# 实验目的

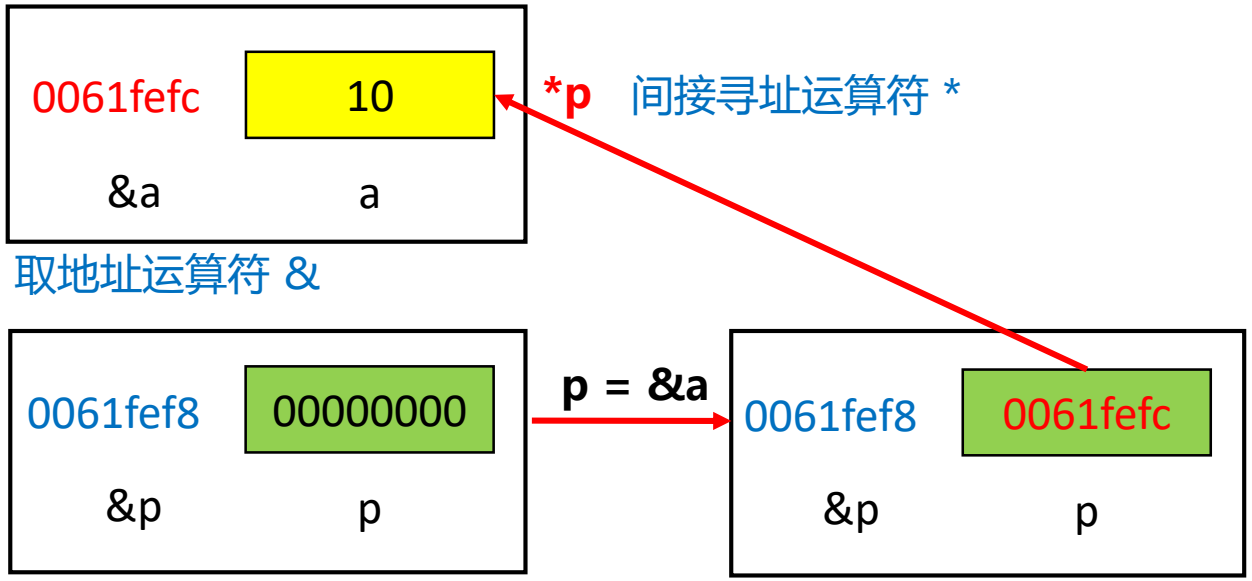
- 理解指针与一维/二维数组的关联关系
- 熟练运用指针实现对一维/二维数组的遍历、访问与操作
- 掌握指针作为函数参数传递一维/二维数组的方法

# 什么是指针?

➢ **指针变量是专门用于存储变量的地址值的变量**

```
int main()
{
    int a = 10;
    int * p = NULL;
    p = &a;
    printf("a is %d, &a is %p\n\n", a, &a);
    printf("p is %p, *p is %d, &p is %p\n", p, *p, &p);
    return 0;
}
```

定义指针变量，并用NULL对其初始化。
永远不要使用未初始化的指针！！！




Watches (new)

| Function arguments | | |
|---|---|---|
| Locals | | |
| a | 10 | |
| p | 0x0 | |
| &a | (int *) 0x61fefc | int * |
| *&a | 10 | int |
| *p | Cannot access memory at address 0x0 | int |
| &p | (int **) 0x61fef8 | int ** |

p = &a

Watches (new)

| Function arguments | | |
|---|---|---|
| Locals | | |
| a | 10 | |
| p | 0x61fefc | |
| &a | (int *) 0x61fefc | int * |
| *&a | 10 | int |
| *p | 10 | int |
| &p | (int **) 0x61fef8 | int ** |

# 为什么使用指针?

- **普通变量做函数参数（按值调用）**

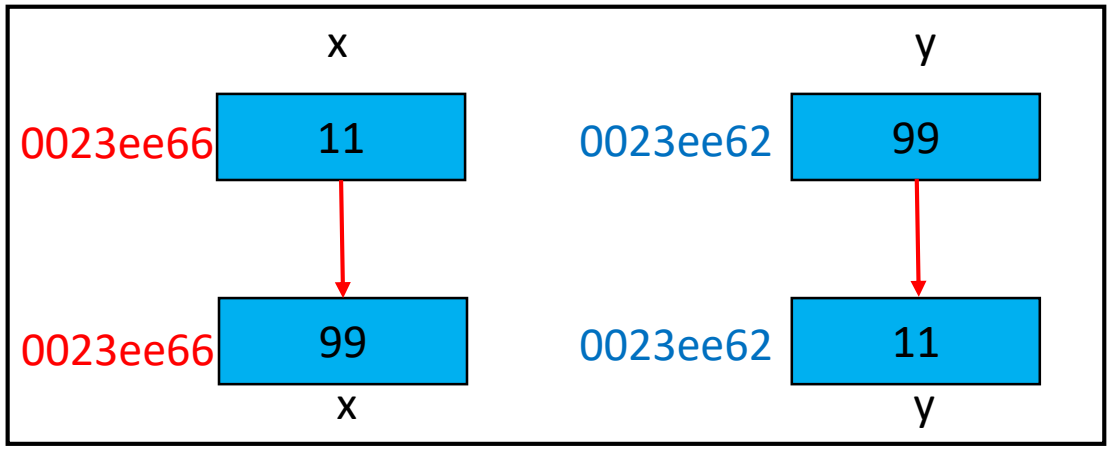
```
void swap(int x, int y)
{
    int t = x;
    x = y;
    y = t;
}

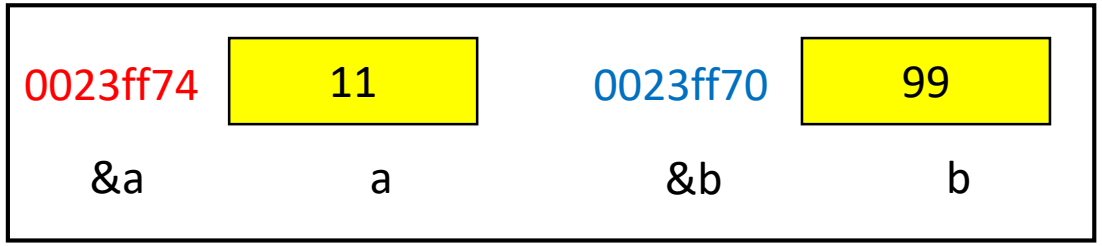
int main()
{
    int a = 11, b = 99;
    printf("Before swap: a = %d, b=%d\n",a,b);

    swap(a, b);
    printf("After swap:  a = %d, b=%d\n",a,b);
}
```

```
D:\CodeTest\output\bin\Debug\output.exe
Before swap: a = 11, b=99
After swap:  a = 11, b=99

Process returned 0 (0x0)   execution time : 4.574 s
Press any key to continue.
```



# 为什么使用指针?

## ➢ 指针变量做函数参数（模拟按引用调用）

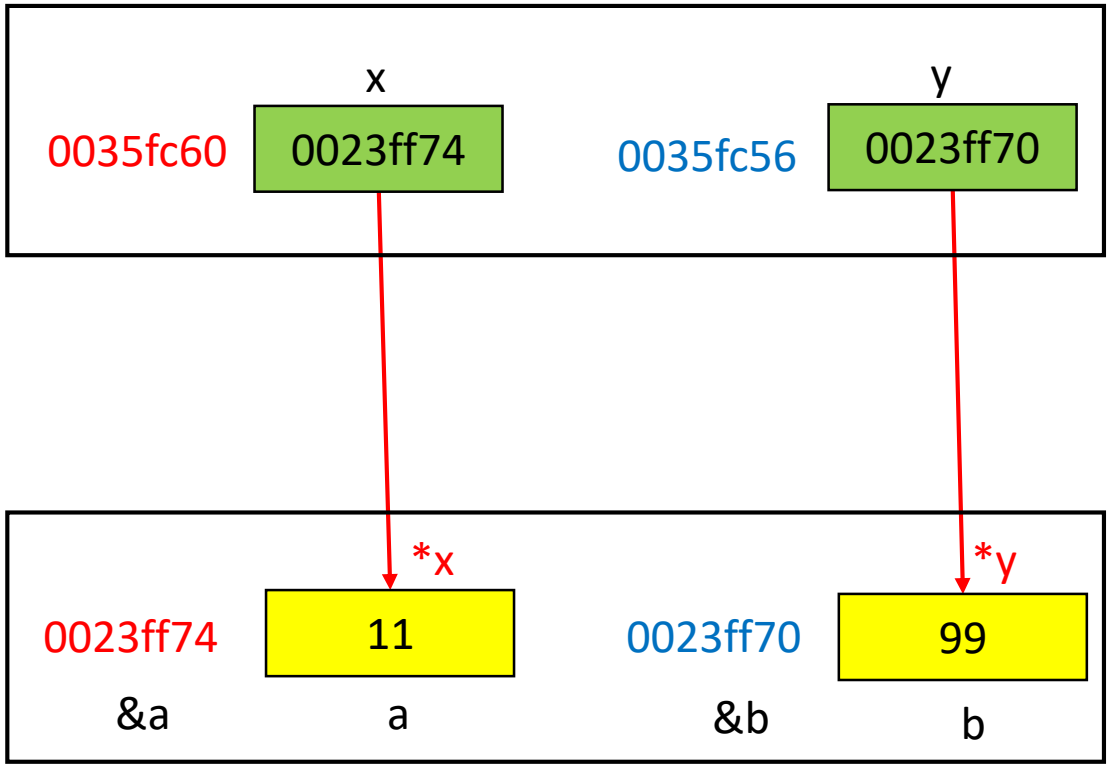
```
void swap(int *x, int *y)
{
    int t = *x;
    *x = *y;
    *y = t;
}

int main()
{
    int a = 11, b = 99;
    printf("Before swap: a = %d, b=%d\n",a,b);

    swap(&a, &b);
    printf("After swap:  a = %d, b=%d\n",a,b);
}
```

```
D:\CodeTest\output\bin\Debug\output.exe
Before swap: a = 11, b=99
After swap:  a = 99, b=11

Process returned 0 (0x0)   execution time : 2.984 s
Press any key to continue.
```



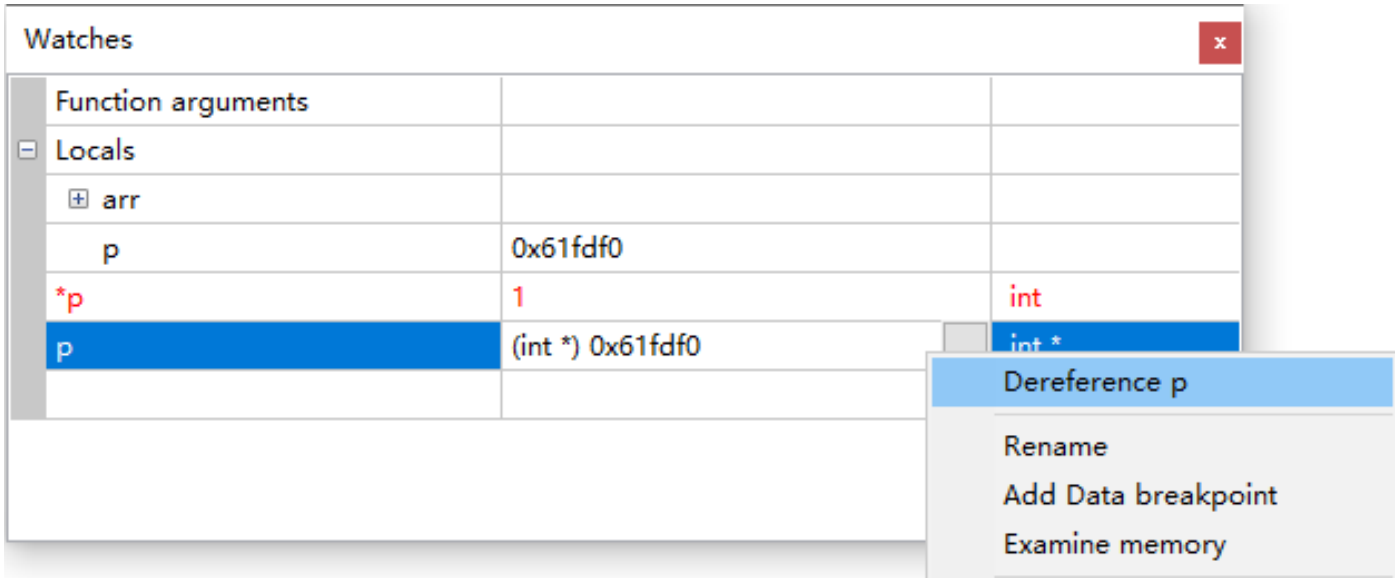
# 调试小技巧

- **如何查看指针指向的变量的值?**

**调试小技巧:**

1. 直接在watches空行键入*p

2. 在watches空行键入p，在此行点击右键，选择“Dereference p”



# 如何用指针操纵一维数组?

- 指针与数组的关系极为密切

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
  int i;
  int score[3] = {70,80,90};

  for ( i =0; i<3; i++)
    printf("score[%d] = %d\n",i,score[i]);
  printf("\n");

  int *p1 = score;
  int *p2 = &score[0];
  int *p3 = p1+1;

  printf("p1 = %p\n",p1);
  printf("p2 = %p\n",p2);
  printf("p3 = %p\n",p3);
  printf("\n");

  printf("*p1 = %d\n",*p1);
  printf("*p2 = %d\n",*p2);
  printf("*p3 = %d\n",*p3);

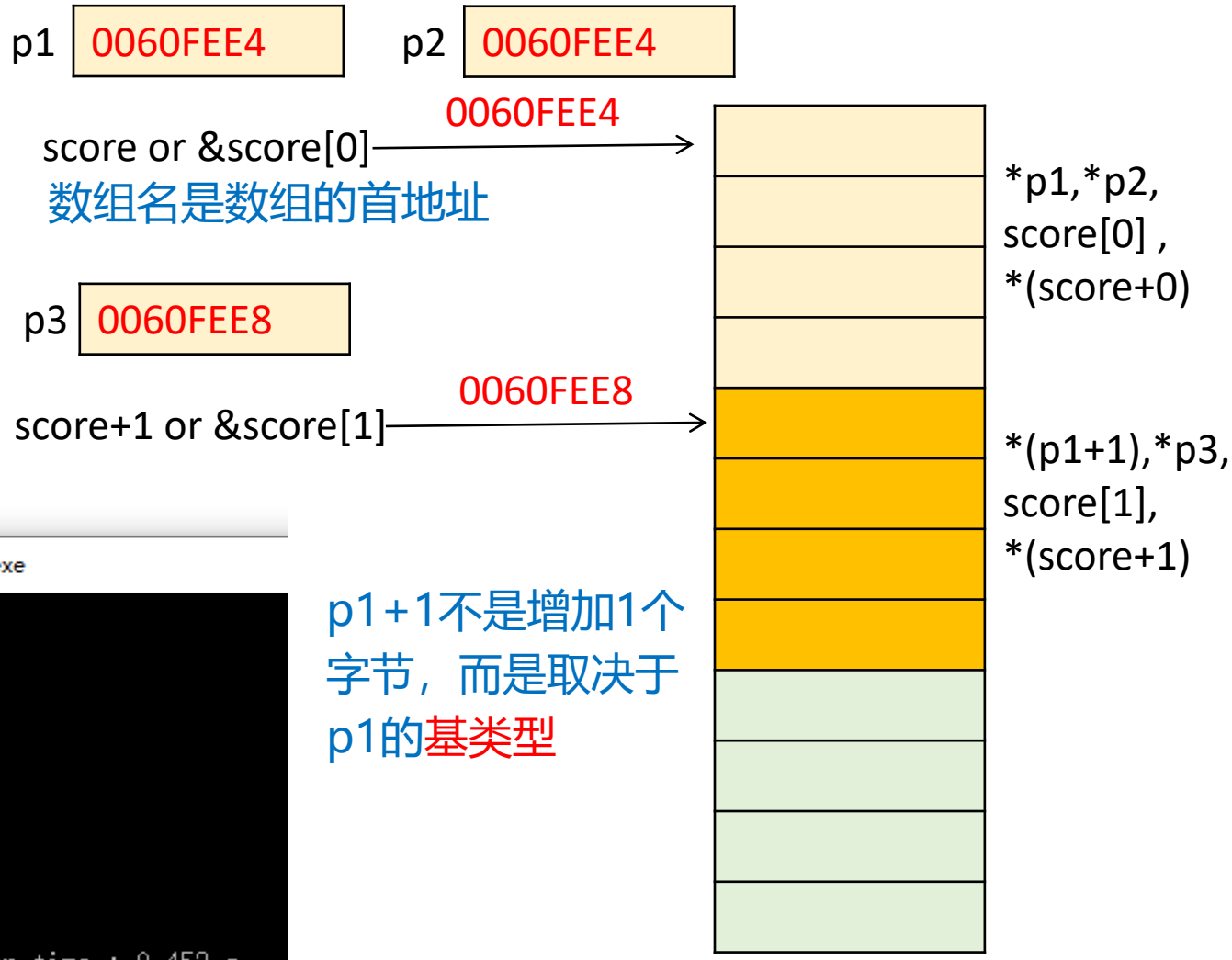
 return 0;
}
```

```
D:\codetest\testouot\bin\Debug\testouot.exe
score[0] = 70
score[1] = 80
score[2] = 90

p1 = 0060FEE4
p2 = 0060FEE4
p3 = 0060FEE8

*p1 = 70
*p2 = 70
*p3 = 80

Process returned 0 (0x0)   execution time : 0.452 s
Press any key to continue.
```




p1+1不是增加1个字节，而是取决于p1的基类型

# 如何用指针操纵二维数组?

➢ **二维数组在内存中是连续存储的，指针操作二维数组主要有两种视角：**

① **行指针：**把二维数组看作一个由一维数组（行）组成的数组。行指针是一个指向一维数组的指针。

**行指针定义：int (*p)[4]**

- p 是一个指针，指向一个包含 4 个 int 的一维数组
- p++步长：**移动一个行**的大小，即 4 * sizeof(int)
- p → 指向一维数组 a[0]
- p + 1 → 第1行指针，指向一维数组 a[1]
- p + i → 第i行指针，指向一维数组 a[i]
- *(p+i) 等价于 a[i] (第 i 行的一维数组本身)
- *(p+i)[j] 或 *(*(p+i)+j) 等价于 a[i][j]

```
int a[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
int (*p)[4] = a
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", *(*(p + i) + j));
    }
    printf("\n");
}
```

a[i] 表示二维数组 a 的第 i 行，是一个包含 4 个 int 的一维数组，类型是 int [4]

# 如何用指针操纵二维数组?

② **列指针**：把二维数组看作一个线性的、连续的内存块。列指针是一个指向数组元素（列）的指针。

**列指针定义：int *q**

- q 是一个普通 int 型指针，指向一个 int 类型的数据
- q++步长：**移动一个元素**的大小，即 sizeof(int)
- q → 指向 a[0][0]
- q+1 → 指向 a[0][1]
- q + i*4 + j → 指向 a[i][j]的指针 (需要知道列数)
- *(q + i*4 + j) 或 q[i*4 + j] 等价于 a[i][j]

```
int a[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
int *q = &a[0][0]
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", *(q + i*4 + j));
    }
    printf("\n");
}
```

# 向函数传递二维数组

## 【例1】输入一个3行4列的二维数组，然后输出这个二维数组的元素值

- 方法一（数组名）：

```
void InputArray(int p[][N], int m, int n)
{
    int i, j;
    for(i = 0; i<m; i++)
    {
        for(j = 0; j<n; j++)
        {
            scanf("%d", &p[i][j]);
        }
    }
}
```

形参声明为二维数组，列数须为常量

```
void OutputArray(int p[][N], int m, int n)
{
    int i, j;
    for(i = 0; i<m; i++)
    {
        for(j = 0; j<n; j++)
        {
            printf("%4d", p[i][j]);
        }
        printf("\n");
    }
}
```

```
InputArray(a, 3, 4);
OutputArray(a, 3, 4);
```

# 向函数传递二维数组

**【例2】输入一个3行4列的二维数组，然后输出这个二维数组的元素值**

- **方法二（行指针）：**

形参声明为二维数组的**行指针**，列数须为常量

```
void InputArray(int (*p)[N], int m, int n)
{
    int i, j;
    for(i = 0; i<m; i++)
    {
        for(j = 0; j<n; j++)
        {
            scanf("%d", *(p+i)+j);
        }
    }
}
```

```
InputArray(a, 3, 4);
OutputArray(a, 3, 4);
```

```
void OutputArray(int (*p)[N], int m, int n)
{
    int i, j;
    for(i = 0; i<m; i++)
    {
        for(j = 0; j<n; j++)
        {
            printf("%4d", *(*(p+i)+j));
        }
        printf("\n");
    }
}
```

# 向函数传递二维数组

## 【例3】输入一个3行4列的二维数组，然后输出这个二维数组的元素值

- 方法三（列指针）：

```
void InputArray(int *p, int m, int n)
{
    int i, j;
    for(i = 0; i<m; i++)
    {
        for(j = 0; j<n; j++)
        {
            scanf("%d", &p[i*n+j]);
        }
    }
}
```

```
InputArray(*a, 3, 4);
OutputArray(*a, 3, 4);
```

形参声明为二维数组的**列指针**，列数可为变量

```
void OutputArray(int *p, int m, int n)
{
    int i, j;
    for(i = 0; i<m; i++)
    {
        for(j = 0; j<n; j++)
        {
            printf("%4d", p[i*n+j]);
        }
        printf("\n");
    }
}
```

# 指针变量作函数参数

## 【例4】计算并输出最高分及相应学生的学号

```
#include <stdio.h>
#include <stdlib.h>
#define N 100   // 定义数组大小
int main()
{
    int score[N], maxScore;
    int n, i;
    long num[N], maxNum;
    printf("How many students?");
    scanf("%d", &n);
    printf("Input student's ID and score:\n");
    for (i=0; i<n; i++)
    {
        scanf("%ld%d", &num[i], &score[i]); /* 字母d前为字母1 */
    }
    FindMax(score, num, n, maxScore, maxNum); /* 按值调用函数 */
    printf("maxScore = %d, maxNum = %ld\n", maxScore, maxNum);
    return 0;
}
```



# 指针变量作函数参数

```
void FindMax(int score[],long num[],int n, int pMaxScore, long pMaxNum)
{
    int    i;
    pMaxScore = score[0];
    pMaxNum = num[0];
    for (i=1; i<n; i++)
    {
        if (score[i] > pMaxScore)
        {
          pMaxScore = score[i];
          pMaxNum = num[i];
        }
    }
}
```

错误原因：普通变量作函数参数按值调用
不能在被调函数中改变相应实参的值

运行结果：

```
How many students?2
Input student's ID and score:
2025001 83
2025002 84
maxScore = 16, maxNum = 0

Process returned 0 (0x0)   execution time : 47.992 s
Press any key to continue.
```

Not Work! Why?

# 指针变量作函数参数

## 修正代码：

```
void FindMax(int score[],long num[],int n, int *pMaxScore, long *pMaxNum)
{
    int     i;
    *pMaxScore = score[0];
    *pMaxNum = num[0];
    for (i=1; i<n; i++)
    {
        if (score[i] > *pMaxScore)
        {
            *pMaxScore = score[i];
            *pMaxNum = num[i];
        }
    }
}
```

重新运行：

```
How many students?2
Input student's ID and score:
2025001 83
2025002 84
maxScore = 84, maxNum = 2025002
```

```
FindMax(score, num, n, &maxScore, &maxNum);
```

# 实验内容

## 3001 一维数组分类、交错合并与去重（50%）

给定长度为 n 的整数数组，例如[1 1 2 2 4 4 8],

- 按原顺序把偶数（含负偶数）提取到偶数组，把奇数（含负奇数）提取到奇数组

偶数组： [2 2 4 4 8]，奇数组：[1 1]。

- 交替从偶数组和奇数组取元素合并成新数组（先取偶后取奇），若一方耗尽[2 1 2 1]，追加另一方全部剩余元素至新数组[2 1 2 1 4 4 8]。
- 合并后的数组去除相邻重复项（连续相同值只保留一个）[2 1 2 1 4 8]

输出合并的去除相邻重复项后的数组

**【输入描述】**

第一行是一个非负整数 n，表示数组长度

第二行是 n 个整数（以空格分隔），表示原始数组元素

**【输出描述】**

合并的去除相邻重复项后的数组元素（以空格分隔）

若n = 0，输出为空。

**【输入示例】**

7

2 2 6 4 4 4 8

**【输出示例】**

2 6 4 8

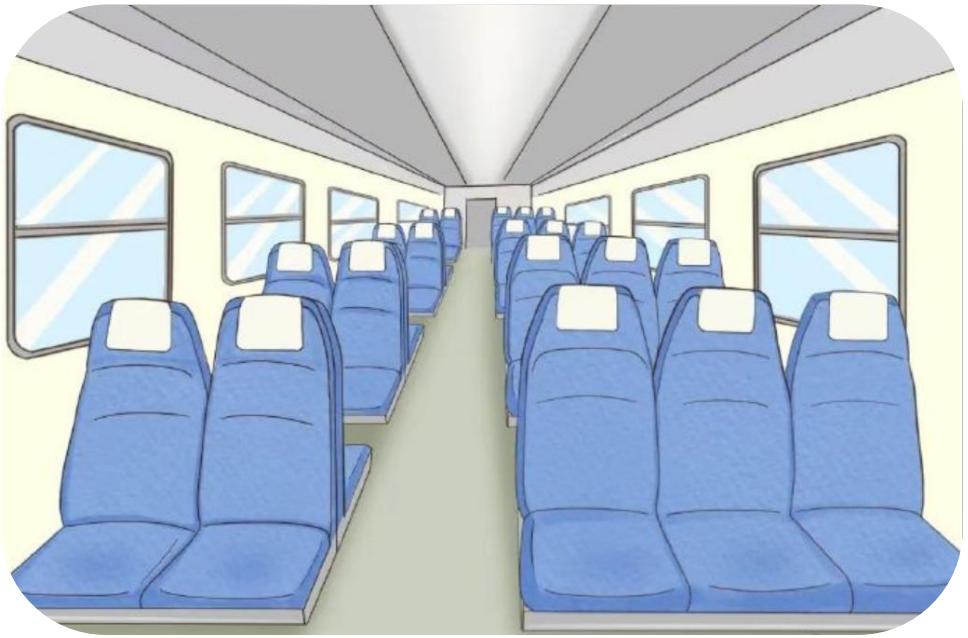
# 实验内容

## 3002 铁路购票系统

**铁路购票系统座位分配算法，用于处理一节车厢的座位分配。**

**规则如下：**假设一节车厢有20排，每一排有5个座位，用 A、B、C、D、F 表示，第一排就是1A、1B、1C、1D、1F，第二排就是2A、2B、2C、2D、2F，以此类推。购票时，每次最多购买5张。

**座位分配策略：**如果这几张票能安排在同一排相邻的座位，则安排在编号最小的相邻座位，否则，安排在编号最小的几个空座位中（不考虑是否相邻）。



# 实验内容

## 3002 铁路购票系统

**【输入示例】**

4　　//购票次数

2 5 4 2　　//每次购票张数

**【输出示例】**

1A 1B

2A 2B 2C 2D 2F

3A 3B 3C 3D

1C 1D

注：每行输出的最后一个座位号后没有空格



# 课程平台使用指南

- 登录网址：http://10.250.48.240:9000，推荐浏览器：Chrome；
- 初始用户名、密码均为学号，登录后请修改；
- 提交截止时间内，可以重新提交作业，不限次数；
- 只要有一次提交通过，即使重新提交代码未通过，也视为access状态；
- 目前仅支持C99，请在本地IDE完成调试后再提交课程平台；
- 代码没有点击“提交”之前，所有内容没有保存，请做好本地备份；
- 代码“提交”，需刷新之后，才可以查看“提交记录”和“已提交代码“；
- 只支持单文件；

# 课程平台使用指南

- 常见oj错误提示：

① Compile Error　编译错误，不符合语法规范

② Wrong Output　表示运行结果不正确

③ Representation error　表示输出格式不正确

④ Runtime error　请检查是否有除零、数组越界、eles语句块没有用{}括起来等

⑤ Nonzero Exit Status　一般表示主函数返回非0值




FEEL THE MEANING OF THE TRIP

# 请同学们开始实验

TAKE YOU ON A TRIP

GO!

哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

探索 从未停止